

Informe Laboratorio 3

Sección 1

Alumno Felipe Maturana Araya
e-mail: felipe.maturana1@mail.udp.cl

Octubre de 2024

Índice

1. Descripción de actividades	2
2. Desarrollo de actividades según criterio de rúbrica	2
2.1. Identifica el algoritmo de hash utilizado al momento de registrarse en el sitio	2
2.2. Identifica el algoritmo de hash utilizado al momento de iniciar sesión	7
2.3. Genera el hash de la contraseña desde la consola del navegador	9
2.4. Intercepta el tráfico login con BurpSuite	10
2.5. Realiza el intento de login	12
2.6. Identifica las políticas de privacidad o seguridad	15
2.7. Demuestra 4 conclusiones sobre la seguridad	15

1. Descripción de actividades

Su objetivo será auditar la implementación de algoritmos hash aplicados a contraseñas en páginas web desde el lado del cliente, así como evaluar la efectividad de estas medidas contra ataques de tipo Pass the Hash (PtH). Para llevar a cabo esta auditoría, deberá registrarse en un sitio web y crear una cuenta, ingresando una contraseña específica para realizar las pruebas.

Al concluir la tarea, es importante que modifique su contraseña por una diferente para garantizar su seguridad.

Dado que la cantidad de sitios chilenos que utilizan hash es limitada, se permite realizar esta tarea en cualquier sitio web a nivel mundial. En este sentido, realice las siguientes actividades:

- Identificación del algoritmo de hash utilizado para las contraseñas al momento del registro en el sitio.
- Identificación del algoritmo de hash utilizado para las contraseñas al momento de iniciar sesión.
- Generación del hash de la contraseña desde la consola del navegador, partiendo de la contraseña en texto plano.
- Interceptación del tráfico de login utilizando BurpSuite desde su equipo.
- Realización de un intento de login, modificando una contraseña incorrecta por el hash obtenido en el punto anterior.
- Descripción de las políticas de privacidad o seguridad relacionadas con las contraseñas, incluyendo un enlace a las mismas.
- Cuatro conclusiones sobre la seguridad o vulnerabilidad de la implementación observada.

2. Desarrollo de actividades según criterio de rúbrica

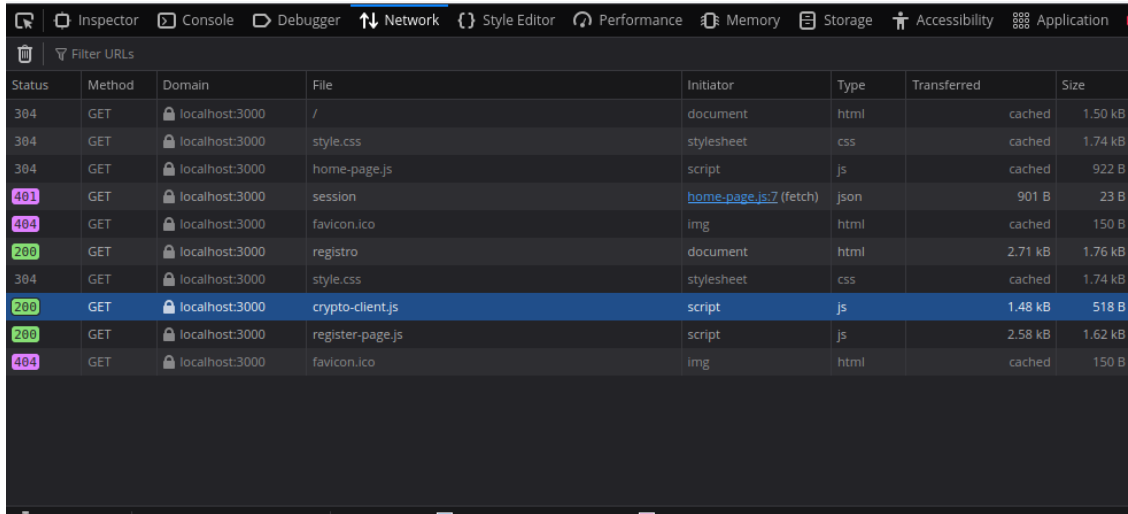
2.1. Identifica el algoritmo de hash utilizado al momento de registrarse en el sitio

Para identificar el algoritmo de hash, primero se accedió a la página de registro del sitio web, como se muestra en la Figura 1.

Iniciar sesión'. At the very bottom of the page, there is a small disclaimer: 'Uso solo con fines de formación en ciberseguridad.'"/>

Figura 1: Interfaz de registro del sitio web de prueba.

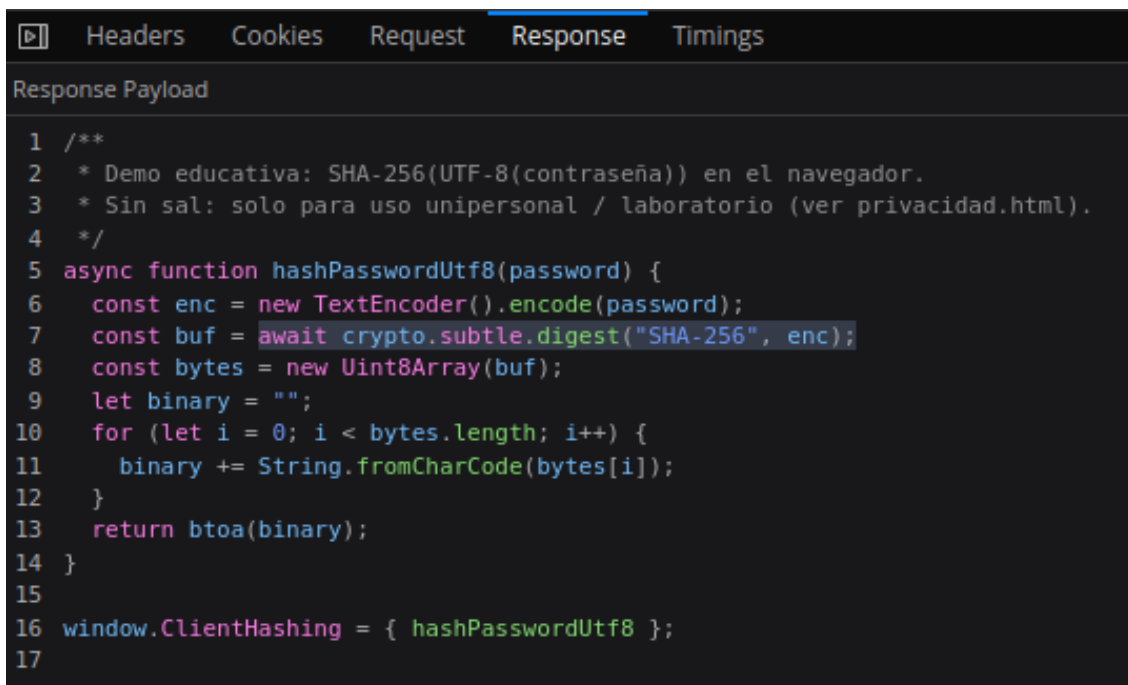
A continuación, se abrieron las herramientas de desarrollador del navegador para inspeccionar los recursos cargados. En la pestaña de red (*Network*), se localizó el archivo `crypto-client.js`, el cual es responsable de la lógica criptográfica en el cliente, tal como se observa en la Figura 2.



Status	Method	Domain	File	Initiator	Type	Transferred	Size
304	GET	localhost:3000	/	document	html	cached	1.50 kB
304	GET	localhost:3000	style.css	stylesheet	css	cached	1.74 kB
304	GET	localhost:3000	home-page.js	script	js	cached	922 B
401	GET	localhost:3000	session	home-page.js:7 (fetch)	json	901 B	23 B
404	GET	localhost:3000	favicon.ico	img	html	cached	150 B
200	GET	localhost:3000	registro	document	html	2.71 kB	1.76 kB
304	GET	localhost:3000	style.css	stylesheet	css	cached	1.74 kB
200	GET	localhost:3000	crypto-client.js	script	js	1.48 kB	518 B
200	GET	localhost:3000	register-page.js	script	js	2.58 kB	1.62 kB
404	GET	localhost:3000	favicon.ico	img	html	cached	150 B

Figura 2: Identificación del archivo `crypto-client.js` en el tráfico de red.

Al analizar el contenido de dicho archivo, se identificó la función `hashPasswordUtf8`, la cual utiliza la API `crypto.subtle.digest` con el algoritmo **SHA-256**. Esta evidencia se presenta en la Figura 3.



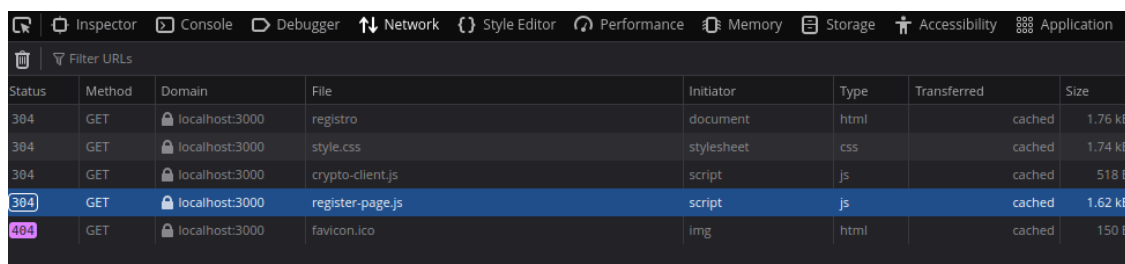
```

1 /**
2  * Demo educativa: SHA-256(UTF-8(contraseña)) en el navegador.
3  * Sin sal: solo para uso unipersonal / laboratorio (ver privacidad.html).
4  */
5 async function hashPasswordUtf8(password) {
6   const enc = new TextEncoder().encode(password);
7   const buf = await crypto.subtle.digest("SHA-256", enc);
8   const bytes = new Uint8Array(buf);
9   let binary = "";
10  for (let i = 0; i < bytes.length; i++) {
11    binary += String.fromCharCode(bytes[i]);
12  }
13  return btoa(binary);
14 }
15
16 window.ClientHashing = { hashPasswordUtf8 };
17

```

Figura 3: Confirmación del uso de SHA-256 en el código fuente.

Posteriormente, se verificó cómo se integra esta función en el flujo de registro. En la Figura 4 se observa la carga del script `register-page.js`, que gestiona el formulario de inscripción.



Status	Method	Domain	File	Initiator	Type	Transferred	Size
304	GET	localhost:3000	registro	document	html	cached	1.76 kB
304	GET	localhost:3000	style.css	stylesheet	css	cached	1.74 kB
304	GET	localhost:3000	crypto-client.js	script	js	cached	518 B
304	GET	localhost:3000	register-page.js	script	js	cached	1.62 kB
404	GET	localhost:3000	favicon.ico	img	html	cached	150 B

Figura 4: Localización del script `register-page.js`.

Finalmente, al revisar el código de `register-page.js`, se confirmó que los datos del formulario son procesados por la función de hashing antes de ser enviados al servidor mediante una petición `fetch`, como se detalla en la Figura 5.



```
1 function () {
2   const form = document.getElementById("form");
3   const msg = document.getElementById("msg");
4   const submit = document.getElementById("submit");
5   if (!form || !msg || !submit) return;
6
7   function show(text, ok) {
8     msg.textContent = text;
9     msg.className = "msg " + (ok ? "ok" : "error");
10    msg.style.display = "block";
11  }
12
13  form.addEventListener("submit", async (e) => {
14    e.preventDefault();
15    msg.style.display = "none";
16    const username = document.getElementById("username").value.trim();
17    const password = document.getElementById("password").value;
18    if (!/^[a-zA-Z0-9._-]{3,32}$/i.test(username)) {
19      show("Nombre de usuario no válido.", false);
20      return;
21    }
22    if (password.length < 8 || password.length > 128) {
23      show("La contraseña debe tener entre 8 y 128 caracteres.", false);
24      return;
25    }
26    submit.disabled = true;
27    try {
28      const verifierBase64 = await window.ClerkHashing.hashPasswordUtf8(
29        password
30      );
31      const res = await fetch("/api/register", {
32        method: "POST",
33        headers: { "Content-Type": "application/json" },
34        credentials: "same-origin",
35        body: JSON.stringify({ username, verifierBase64 }),
36      });
37      const data = await res.json().catch(() => ({}));
38      if (res.ok) {
39        show("Cuenta creada. Ya puedes iniciar sesión.", true);
40        form.reset();
41      } else {
42        show(data.error || "No se pudo registrar.", false);
43      }
44    } catch (_err) {
45      show("Error de red o del navegador.", false);
46    }
47  });
48 }
```

Figura 5: Llamada a la función de hashing en el proceso de registro.

2.2. Identifica el algoritmo de hash utilizado al momento de iniciar sesión

Para el proceso de inicio de sesión, se siguió una metodología similar. Primero se navegó a la interfaz de login, visible en la Figura 6.

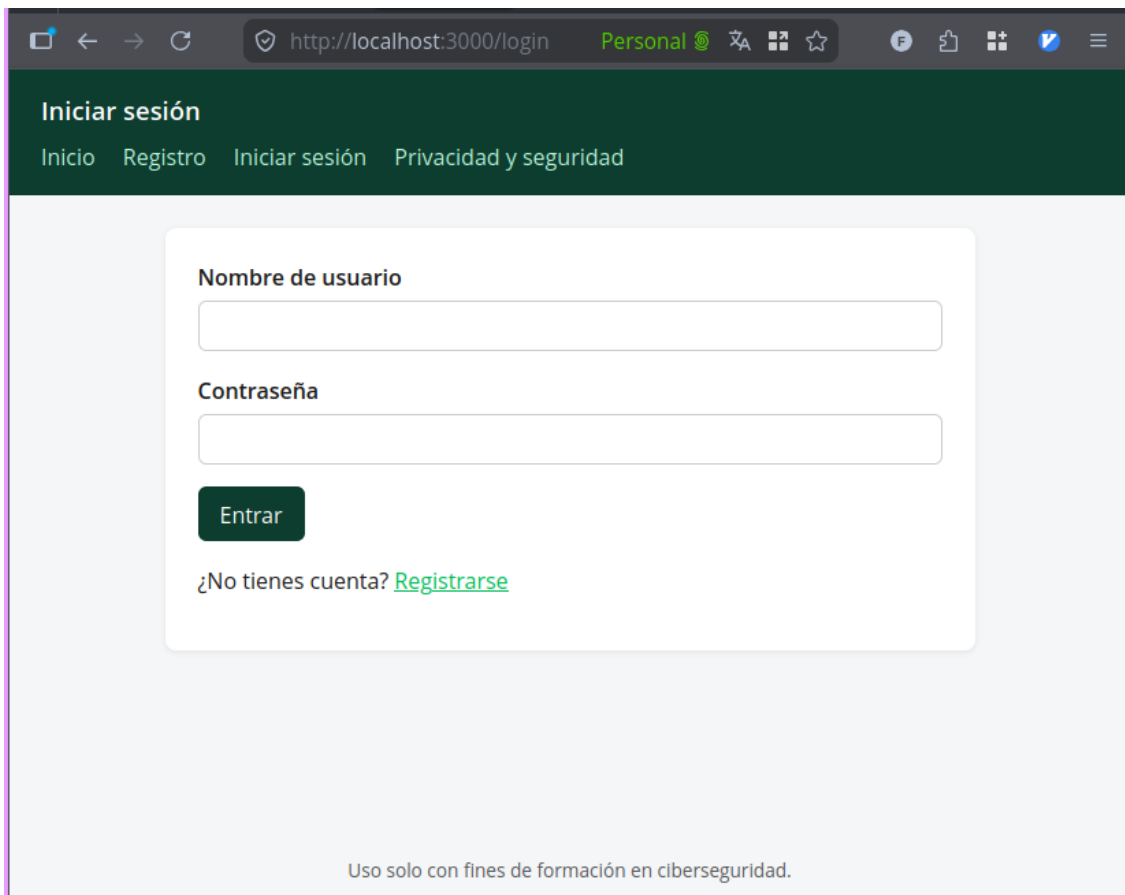


Figura 6: Interfaz de inicio de sesión.

Se inspeccionó nuevamente el tráfico de red para validar qué scripts controlan este formulario, identificando el archivo `login-page.js`, como se muestra en la Figura 7.

Status	Method	Domain	File	Initiator	Type	Transferred	Size
304	GET	localhost:3000	login	document	html	cached	1.77 kB
304	GET	localhost:3000	style.css	stylesheet	css	cached	1.74 kB
304	GET	localhost:3000	crypto-client.js	script	js	cached	518 B
304	GET	localhost:3000	login-page.js	script	js	cached	1.67 kB
404	GET	localhost:3000	favicon.ico	img	html	cached	150 B

Figura 7: Detección del script `login-page.js`.

Al examinar el código de la respuesta, se comprobó que el sitio web reutiliza la misma lógica de SHA-256 definida en el cliente para procesar la contraseña del usuario antes de la autenticación. Este paso final de verificación se ilustra en la Figura 8.



```

1 function () {
2   const form = document.getElementById("form");
3   const msg = document.getElementById("msg");
4   const submit = document.getElementById("submit");
5   if (!form || !msg || !submit) return;
6
7   function show(text, ok) {
8     msg.textContent = text;
9     msg.className = "msg " + (ok ? "ok" : "error");
10    msg.style.display = "block";
11  }
12
13  form.addEventListener("submit", async (e) => {
14    e.preventDefault();
15    msg.style.display = "none";
16    const username = document.getElementById("username").value.trim();
17    const password = document.getElementById("password").value;
18    if (!/^[a-zA-Z0-9._-]{3,32}$/i.test(username)) {
19      show("Nombre de usuario no válido.", false);
20      return;
21    }
22    if (password.length < 8 || password.length > 128) {
23      show("La contraseña debe tener entre 8 y 128 caracteres.", false);
24      return;
25    }
26    submit.disabled = true;
27    try {
28      const verifierBase64 = await window.ClientHashing.hashPasswordUtf8(
29        password
30      );
31      const res = await fetch("/api/login", {
32        method: "POST",
33        headers: { "Content-Type": "application/json" },
34        credentials: "same-origin",
35        body: JSON.stringify({ username, verifierBase64 }),
36      });
37      const data = await res.json().catch(() => ({}));
38      if (res.ok) {
39        show("Sesión iniciada correctamente.", true);
40        setTimeout(() => {
41          window.location.href = "/";
42        }, 600);
43      } else {
44        show(data.error || "Credenciales incorrectas.", false);
45      }
46    } catch (_err) {
47      show("Error de red o del navegador.", false);
48    } finally {
49      submit.disabled = false;
50    }
51  });
52 }

```

Figura 8: Uso consistente de SHA-256 durante el inicio de sesión.

2.3. Genera el hash de la contraseña desde la consola del navegador

Para emular el comportamiento de la aplicación y obtener el valor real que se envía al servidor, se replicó la lógica criptográfica utilizando la consola del desarrollador del navegador. El procedimiento consistió en codificar la contraseña en texto plano a UTF-8, aplicar la función de resumen SHA-256 utilizando la API nativa `crypto.subtle.digest`, y finalmente convertir el buffer de bytes resultante a una cadena en formato Base64.

El código ejecutado para realizar esta transformación es el siguiente:

```
(async () => {  
  const p = "pepe1234";  
  const enc = new TextEncoder().encode(p);  
  const buf = await crypto.subtle.digest("SHA-256", enc);  
  const b64 = btoa(String.fromCharCode(...new Uint8Array(buf)));  
  console.log("Tu hash en Base64 es:", b64);  
})();
```

La Figura 9 demuestra la ejecución exitosa de este script en la consola, retornando el hash en Base64 que representa la contraseña "pepe1234".

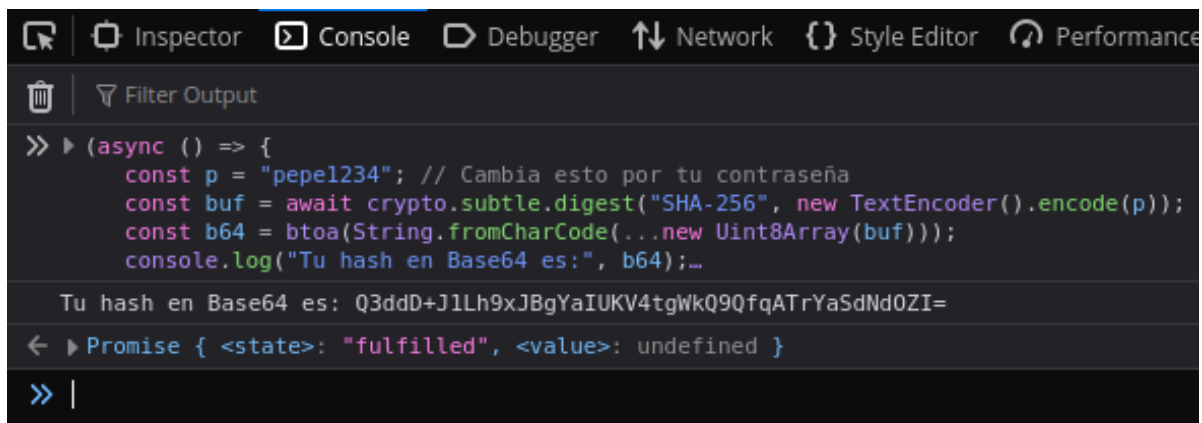


Figura 9: Ejecución del script en consola y obtención del hash en Base64.

2.4. Intercepta el tráfico login con BurpSuite

Para auditar la comunicación entre el cliente y el servidor, se procedió a configurar un proxy de interceptación utilizando la herramienta Burp Suite. El primer paso consistió en asegurar que el interceptor estuviera activo en la pestaña **Proxy**, como se observa en la Figura 10.

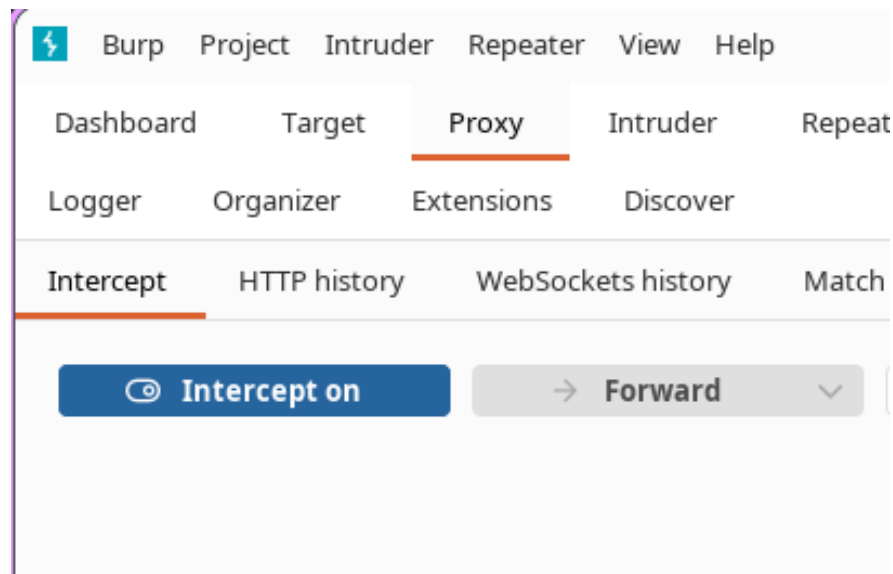


Figura 10: Activación del modo Intercept en Burp Suite.

Posteriormente, se navegó a la página de inicio de sesión de la aplicación. Se utilizó la cuenta creada anteriormente con el nombre de usuario "**pepe**" (cuya contraseña real es "**pepe1234**"). Sin embargo, con el objetivo de capturar una petición fallida para su posterior manipulación, se completó el formulario ingresando una clave deliberadamente incorrecta: "**clavefalsa**", tal como se muestra en la Figura 11.



localhost:3000/login

Iniciar sesión Privacidad y seguridad

Nombre de usuario

pepe

Contraseña

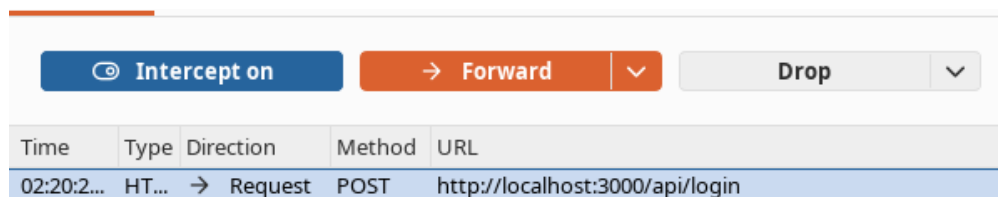
.....

Entrar

¿No tienes cuenta? [Registrarse](#)

Figura 11: Formulario de inicio de sesión completado con credenciales incorrectas.

Al presionar el botón de inicio de sesión, Burp Suite capturó la petición HTTP saliente. En la Figura 12 se puede apreciar la captura de la petición POST dirigida al endpoint `/api/login`, la cual quedó retenida en el proxy a la espera de ser procesada o modificada.



Time	Type	Direction	Method	URL
02:20:2...	HT...	→ Request	POST	http://localhost:3000/api/login

Figura 12: Petición POST interceptada por Burp Suite.

2.5. Realiza el intento de login

Con la petición interceptada, se inició la fase de explotación del ataque Pass the Hash. Primero, se inspeccionó el cuerpo de la petición original en formato JSON. En la Figura 13 se observa el parámetro `verifierBase64`, el cual contiene el hash SHA-256 correspondiente a la contraseña incorrecta ("clavefalsa").

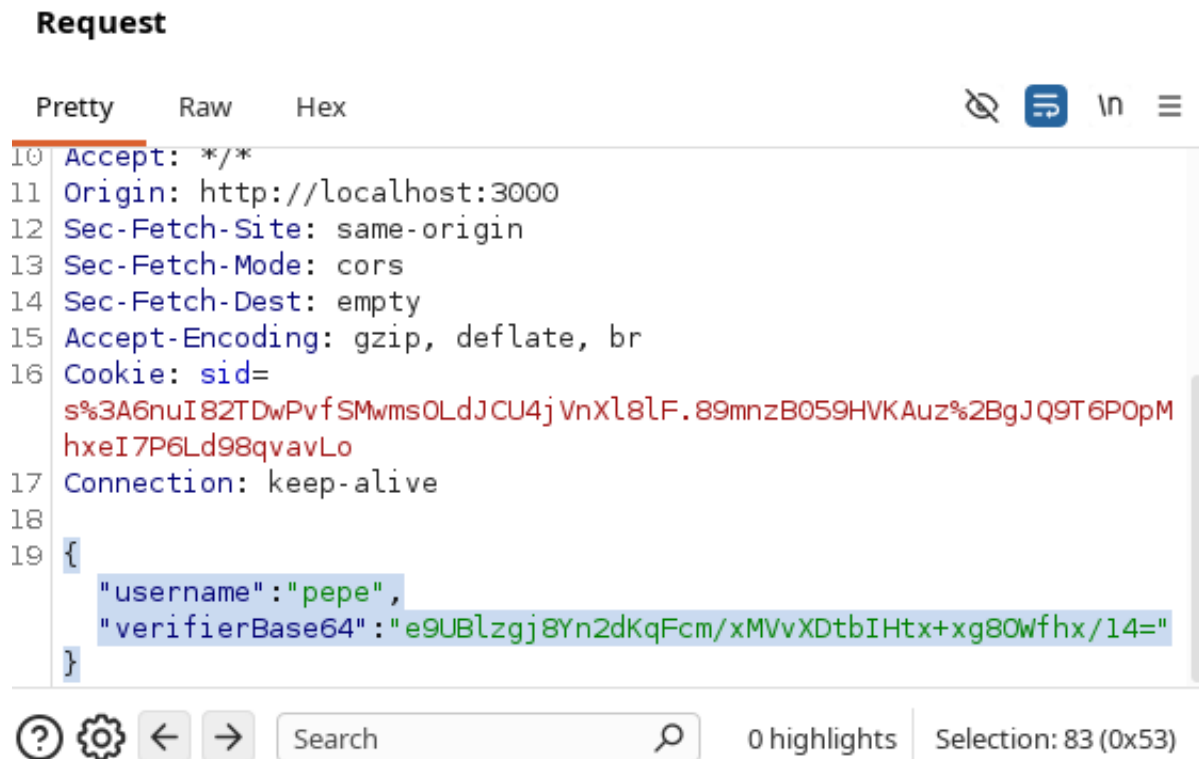
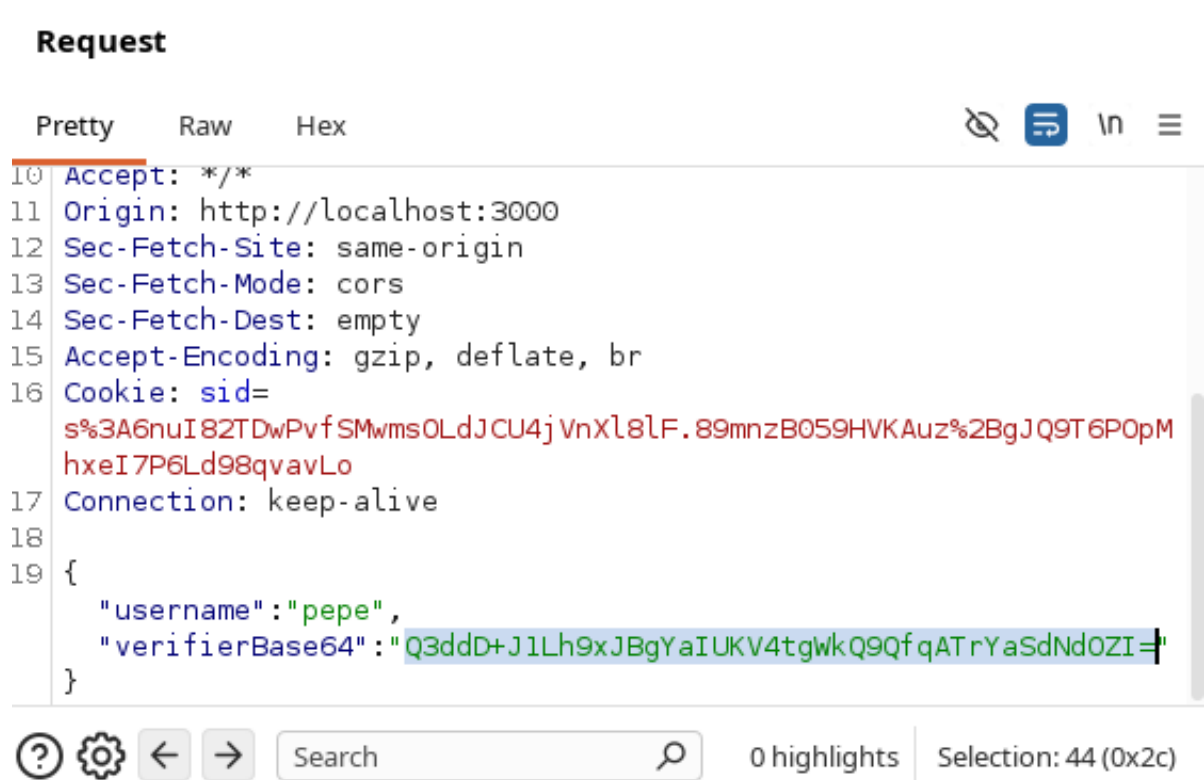


Figura 13: Payload original con el hash de la contraseña incorrecta.

A continuación, se procedió a modificar este parámetro manualmente dentro de Burp Suite. Se reemplazó el valor original por el hash Base64 obtenido previamente en la sección 2.3 del informe (el hash legítimo de "pepe1234"). La Figura 14 muestra el payload ya manipulado antes de ser reenviado (*forwarded*) al servidor.



```
Request
Pretty Raw Hex
10 Accept: */*
11 Origin: http://localhost:3000
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Dest: empty
15 Accept-Encoding: gzip, deflate, br
16 Cookie: sid=
    s%3A6nuI82TDwPvfSMwmsOLdJCU4jVnXl8lF.89mnzB059HVKAuZ%2BgJQ9T6P0pM
    hxeI7P6Ld98qvavLo
17 Connection: keep-alive
18
19 {
    "username": "pepe",
    "verifierBase64": "Q3ddD+J1Lh9xJBgYaIUkV4tgWkQ9QfqATrYaSdNdOZI="
}
```

Figura 14: Payload manipulado con el hash real inyectado manualmente.

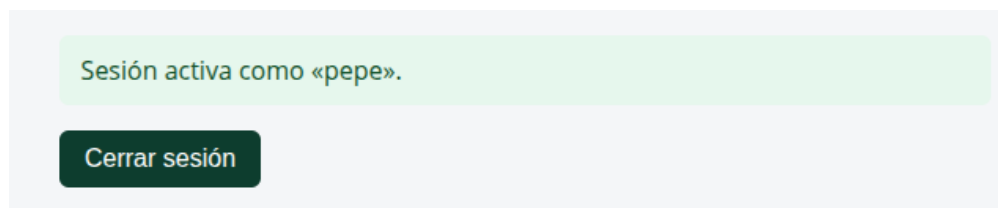
Una vez enviada la petición modificada, se regresó al navegador para verificar el resultado. A pesar de haber ingresado "clavefalsa" en el formulario, el servidor validó el hash inyectado y permitió el acceso, como se evidencia en el mensaje de éxito de la Figura 15.



The image shows a login interface with a light green background. At the top, a green message box says "Sesión iniciada correctamente." Below this, there are two input fields: "Nombre de usuario" containing the text "pepe" and "Contraseña" containing a series of dots. A dark green "Entrar" button is positioned below the password field. At the bottom, there is a link that says "¿No tienes cuenta? [Registrarse](#)".

Figura 15: Confirmación de sesión iniciada exitosamente tras el ataque.

Finalmente, la Figura 16 muestra la página de inicio con la sesión activa del usuario "pepe", demostrando que el conocimiento del hash es suficiente para suplantar la identidad del usuario sin poseer la contraseña original.



The image shows a session active message box with a light green background. The text inside says "Sesión activa como «pepe»." Below the message box is a dark green button labeled "Cerrar sesión".

Figura 16: Vista de la sesión activa en la página principal.

2.6. Identifica las políticas de privacidad o seguridad

Al revisar la sección correspondiente del sitio web (disponible en `http://localhost:3000/privacidad`), se constata que la plataforma declara explícitamente su mecanismo de manejo de credenciales. La política informa a los usuarios que las contraseñas no se transmiten en texto plano, sino que son procesadas localmente mediante el algoritmo SHA-256 antes de su envío.

Desde la perspectiva de una auditoría de ciberseguridad, destaca la inusual transparencia del documento al admitir las debilidades de su propia arquitectura. La política reconoce explícitamente la ausencia de "sal" (salt) criptográfica y advierte que la interceptación del resumen permitiría su reutilización, validando teóricamente la viabilidad del ataque *Pass the Hash* ejecutado en la sección anterior. Asimismo, detalla que el servidor únicamente conserva el nombre de usuario, el verificador SHA-256 y una cookie de sesión `HttpOnly`. Aunque la declaración cumple con el principio de transparencia informativa, a nivel técnico confirma un diseño inherentemente inseguro para entornos de producción, ya que transforma el hash interceptable en el equivalente a una contraseña en texto plano.

2.7. Demuestra 4 conclusiones sobre la seguridad

Tras la auditoría realizada, se presentan las siguientes conclusiones técnicas:

1. **Vulnerabilidad del hash como equivalente a texto plano:** El laboratorio demostró que, al hashear la contraseña solo en el cliente, el hash resultante se convierte en el nuevo secreto de autenticación. Como el servidor no pide la contraseña original sino este código, cualquier atacante que intercepte el hash puede entrar a la cuenta sin conocer jamás la clave real del usuario.
2. **Efectividad del ataque Pass the Hash (PtH):** La manipulación exitosa de la petición con Burp Suite confirmó que el sistema es vulnerable a ataques de reuso de credenciales. Al inyectar un hash válido en una petición fallida, se comprobó que el servidor no tiene mecanismos para distinguir entre un cálculo legítimo del navegador y una inyección manual de un tercero.
3. **Riesgo por falta de sal (Salt):** La ausencia de sal criptográfica personalizada por usuario facilita ataques de diccionario o tablas precomputadas. Si dos usuarios eligen la misma contraseña, sus hashes en la base de datos serán idénticos, lo que permite a un atacante comprometer múltiples cuentas simultáneamente una vez que descifra un solo valor.
4. **Inseguridad del hashing de un solo paso:** El uso de un único SHA-256 es insuficiente para proteger credenciales modernas. Una implementación segura debería realizar el hashing principalmente en el servidor utilizando funciones lentas y costosas (como Argon2 o bcrypt) diseñadas para resistir ataques de fuerza bruta, en lugar de confiar únicamente en un resumen rápido ejecutado en el navegador.